Transformer From Scratch in PyTorch

This notebook builds a Transformer encoder-decoder model from first principles using PyTorch. It includes multi-head attention, positional encoding, masking, encoder and decoder blocks, a full Transformer module, and a small CUDA-ready demo training loop.

1. Imports and device setup

```
import math
from dataclasses import dataclass

import torch
import torch.nn as nn
import torch.nn.functional as F


device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

2. Core Transformer building blocks

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_len: int = 5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len,
dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2,
dtype=torch.float) * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe.unsqueeze(0))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x + self.pe[:, : x.size(1)]


class MultiHeadAttention(nn.Module):
    def __init__(self, d_model: int, num_heads: int, dropout: float =
0.1):
        super().__init__()
        if d_model % num_heads != 0:
            raise ValueError("d_model must be divisible by num_heads")

        self.d_model = d_model
        self.num_heads = num_heads
```

```
        self.head_dim = d_model // num_heads

        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def _split_heads(self, x: torch.Tensor) -> torch.Tensor:
        batch_size, seq_len, _ = x.shape
        x = x.view(batch_size, seq_len, self.num_heads, self.head_dim)
        return x.transpose(1, 2)

    def forward(self, query: torch.Tensor, key: torch.Tensor, value:
torch.Tensor, mask: torch.Tensor | None = None) -> torch.Tensor:
        batch_size = query.size(0)

        q = self._split_heads(self.q_proj(query))
        k = self._split_heads(self.k_proj(key))
        v = self._split_heads(self.v_proj(value))

        scores = torch.matmul(q, k.transpose(-2, -1)) /
math.sqrt(self.head_dim)

        if mask is not None:
            if mask.dim() == 3:
                mask = mask.unsqueeze(1)
            scores = scores.masked_fill(~mask.bool(), float("-inf"))

        attention = torch.softmax(scores, dim=-1)
        attention = self.dropout(attention)
        output = torch.matmul(attention, v)

        output = output.transpose(1, 2).contiguous().view(batch_size,
-1, self.d_model)
        return self.out_proj(output)


class FeedForward(nn.Module):
    def __init__(self, d_model: int, d_ff: int, dropout: float = 0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

```python
class EncoderLayer(nn.Module):
    def __init__(self, d_model: int, num_heads: int, d_ff: int,
dropout: float = 0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads,
dropout)
        self.ffn = FeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor, src_mask: torch.Tensor | None =
None) -> torch.Tensor:
        attn_out = self.self_attn(x, x, x, src_mask)
        x = self.norm1(x + self.dropout(attn_out))
        ff_out = self.ffn(x)
        x = self.norm2(x + self.dropout(ff_out))
        return x


class DecoderLayer(nn.Module):
    def __init__(self, d_model: int, num_heads: int, d_ff: int,
dropout: float = 0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads,
dropout)
        self.cross_attn = MultiHeadAttention(d_model, num_heads,
dropout)
        self.ffn = FeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(
        self,
        x: torch.Tensor,
        enc_out: torch.Tensor,
        src_mask: torch.Tensor | None = None,
        tgt_mask: torch.Tensor | None = None,
    ) -> torch.Tensor:
        self_attn_out = self.self_attn(x, x, x, tgt_mask)
        x = self.norm1(x + self.dropout(self_attn_out))
        cross_attn_out = self.cross_attn(x, enc_out, enc_out,
src_mask)
        x = self.norm2(x + self.dropout(cross_attn_out))
        ff_out = self.ffn(x)
```

```
        x = self.norm3(x + self.dropout(ff_out))
        return x
```

3. Full Transformer and masking utilities

```
class Transformer(nn.Module):
    def __init__(
        self,
        src_vocab_size: int,
        tgt_vocab_size: int,
        d_model: int = 256,
        num_layers: int = 4,
        num_heads: int = 8,
        d_ff: int = 512,
        dropout: float = 0.1,
        max_len: int = 500,
    ):
        super().__init__()
        self.src_embedding = nn.Embedding(src_vocab_size, d_model)
        self.tgt_embedding = nn.Embedding(tgt_vocab_size, d_model)
        self.positional_encoding = PositionalEncoding(d_model,
max_len)
        self.encoder_layers = nn.ModuleList(
            [EncoderLayer(d_model, num_heads, d_ff, dropout) for _ in
range(num_layers)]
        )
        self.decoder_layers = nn.ModuleList(
            [DecoderLayer(d_model, num_heads, d_ff, dropout) for _ in
range(num_layers)]
        )
        self.dropout = nn.Dropout(dropout)
        self.fc_out = nn.Linear(d_model, tgt_vocab_size)
        self.d_model = d_model

    def encode(self, src: torch.Tensor, src_mask: torch.Tensor | None
= None) -> torch.Tensor:
        x = self.src_embedding(src) * math.sqrt(self.d_model)
        x = self.positional_encoding(x)
        x = self.dropout(x)
        for layer in self.encoder_layers:
            x = layer(x, src_mask)
        return x

    def decode(
        self,
        tgt: torch.Tensor,
        enc_out: torch.Tensor,
        src_mask: torch.Tensor | None = None,
        tgt_mask: torch.Tensor | None = None,
```

```
    ) -> torch.Tensor:
        x = self.tgt_embedding(tgt) * math.sqrt(self.d_model)
        x = self.positional_encoding(x)
        x = self.dropout(x)
        for layer in self.decoder_layers:
            x = layer(x, enc_out, src_mask, tgt_mask)
        return x

    def forward(
        self,
        src: torch.Tensor,
        tgt: torch.Tensor,
        src_mask: torch.Tensor | None = None,
        tgt_mask: torch.Tensor | None = None,
    ) -> torch.Tensor:
        enc_out = self.encode(src, src_mask)
        dec_out = self.decode(tgt, enc_out, src_mask, tgt_mask)
        return self.fc_out(dec_out)


def create_src_mask(src: torch.Tensor, pad_idx: int) -> torch.Tensor:
    return (src != pad_idx).unsqueeze(1).unsqueeze(2)


def create_tgt_mask(tgt: torch.Tensor, pad_idx: int) -> torch.Tensor:
    batch_size, tgt_len = tgt.shape
    pad_mask = (tgt != pad_idx).unsqueeze(1).unsqueeze(2)
    causal_mask = torch.tril(torch.ones((tgt_len, tgt_len),
device=tgt.device)).bool()
    causal_mask = causal_mask.unsqueeze(0).unsqueeze(1)
    return pad_mask & causal_mask
```

4. Sanity check and CUDA-ready training demo

```
@dataclass
class Config:
    src_vocab_size: int = 100
    tgt_vocab_size: int = 100
    pad_idx: int = 0
    d_model: int = 256
    num_layers: int = 2
    num_heads: int = 8
    d_ff: int = 512
    dropout: float = 0.1
    batch_size: int = 16
    src_len: int = 12
    tgt_len: int = 12
    lr: float = 3e-4
```

```
config = Config()
model = Transformer(
    src_vocab_size=config.src_vocab_size,
    tgt_vocab_size=config.tgt_vocab_size,
    d_model=config.d_model,
    num_layers=config.num_layers,
    num_heads=config.num_heads,
    d_ff=config.d_ff,
    dropout=config.dropout,
).to(device)

optimizer = torch.optim.Adam(model.parameters(), lr=config.lr)
criterion = nn.CrossEntropyLoss(ignore_index=config.pad_idx)

model.train()

src = torch.randint(1, config.src_vocab_size, (config.batch_size,
config.src_len), device=device)
tgt = torch.randint(1, config.tgt_vocab_size, (config.batch_size,
config.tgt_len), device=device)

src_mask = create_src_mask(src, config.pad_idx)
tgt_input = tgt[:, :-1]
tgt_target = tgt[:, 1:]
tgt_mask = create_tgt_mask(tgt_input, config.pad_idx)

logits = model(src, tgt_input, src_mask, tgt_mask)
loss = criterion(logits.reshape(-1, logits.size(-1)),
tgt_target.reshape(-1))

optimizer.zero_grad()
loss.backward()
optimizer.step()

print("Transformer forward pass complete")
print(f"Logits shape: {tuple(logits.shape)}")
print(f"Loss: {loss.item():.4f}")
print("CUDA available:", torch.cuda.is_available())
```

```
Transformer forward pass complete
Logits shape: (16, 11, 100)
Loss: 4.7391
CUDA available: True
```

5. Tiny copy-task training demo

This extra demo shows the model learning a simple sequence-copy task instead of only doing one random forward pass.

```
def generate_copy_batch(batch_size: int, seq_len: int, vocab_size:
int, pad_idx: int, device: torch.device):
    tokens = torch.randint(1, vocab_size, (batch_size, seq_len),
device=device)
    src = tokens.clone()
    tgt = tokens.clone()
    src_mask = create_src_mask(src, pad_idx)
    tgt_input = tgt[:, :-1]
    tgt_target = tgt[:, 1:]
    tgt_mask = create_tgt_mask(tgt_input, pad_idx)
    return src, tgt_input, tgt_target, src_mask, tgt_mask


def greedy_decode(model: nn.Module, src: torch.Tensor, max_len: int,
start_token: int, pad_idx: int) -> torch.Tensor:
    model.eval()
    with torch.no_grad():
        src_mask = create_src_mask(src, pad_idx)
        enc_out = model.encode(src, src_mask)
        generated = torch.full((src.size(0), 1), start_token,
dtype=torch.long, device=src.device)

        for _ in range(max_len - 1):
            tgt_mask = create_tgt_mask(generated, pad_idx)
            logits = model.decode(generated, enc_out, src_mask,
tgt_mask)
            next_token = model.fc_out(logits[:, -1]).argmax(dim=-1,
keepdim=True)
            generated = torch.cat([generated, next_token], dim=1)

    return generated


copy_model = Transformer(
    src_vocab_size=config.src_vocab_size,
    tgt_vocab_size=config.tgt_vocab_size,
    d_model=config.d_model,
    num_layers=config.num_layers,
    num_heads=config.num_heads,
    d_ff=config.d_ff,
    dropout=config.dropout,
).to(device)

copy_optimizer = torch.optim.Adam(copy_model.parameters(),
lr=config.lr)
copy_criterion = nn.CrossEntropyLoss(ignore_index=config.pad_idx)

copy_model.train()
for step in range(1, 11):
    src, tgt_input, tgt_target, src_mask, tgt_mask =
```

```
generate_copy_batch(
        config.batch_size,
        config.tgt_len,
        config.src_vocab_size,
        config.pad_idx,
        device,
    )

    output = copy_model(src, tgt_input, src_mask, tgt_mask)
    copy_loss = copy_criterion(output.reshape(-1, output.size(-1)),
tgt_target.reshape(-1))

    copy_optimizer.zero_grad()
    copy_loss.backward()
    copy_optimizer.step()

    if step == 1 or step % 5 == 0:
        print(f"Step {step:02d} | Copy loss: {copy_loss.item():.4f}")

sample_src, _, sample_tgt_target, _, _ = generate_copy_batch(1,
config.tgt_len, config.src_vocab_size, config.pad_idx, device)
sample_pred = greedy_decode(copy_model, sample_src, config.tgt_len -
1, start_token=sample_src[:, 0:1].item(), pad_idx=config.pad_idx)

print("Source tokens:", sample_src.squeeze(0).tolist())
print("Target tokens:", sample_tgt_target.squeeze(0).tolist())
print("Predicted tokens:", sample_pred.squeeze(0).tolist())
```

```
Step 01 | Copy loss: 4.8522
Step 05 | Copy loss: 4.7789
Step 10 | Copy loss: 4.7512
Source tokens: [43, 4, 85, 42, 17, 94, 44, 42, 78, 69, 37, 79]
Target tokens: [4, 85, 42, 17, 94, 44, 42, 78, 69, 37, 79]
Predicted tokens: [43, 1, 48, 93, 11, 11, 11, 11, 11, 11, 11]
```